МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»

Інститут комп’ютерних технологій, автоматики та метрології

Кафедра безпеки інформаційних технологій

КУРСОВИЙ ПРОЄКТ

з дисципліни

«ОСНОВИ ІНТЕРНЕТУ РЕЧЕЙ ТА АНАЛІТИКА ВЕЛИКИХ ДАНИХ»

на тему

«Проєктування системи збору та аналітики великих даних IoT-інфраструктури з виявленням аномалій та ознак компрометації пристроїв»

(варіант № [номер вашого варіанта])

Студента (ки) групи КБ-[номер групи]
[Прізвище Ім’я По батькові]
_____ (підпис)

Керівник курсового проєкту:
[вчений ступінь, вчене звання, посада]
[Прізвище Ім’я По батькові]

Львів 2026

КАРТКА РЕЗУЛЬТАТІВ

Поле	Значення
Варіант	*0 (навчальний набір `tuning.parquet`)*
Кількість записів	*1 157 147*
Пристроїв у маніфесті	*51*
Пристроїв у даних	*52*
SHA-256 переліку пристроїв	*1a33058d845b45c2ce0ddff78318cad7 cab1f1b8c9a25aed2e3cdbafc84b7098*
Записів за показниками	*co2 29 050; door_state 82 111; flow_lpm 39 270; free_heap 11 947; humidity 176 815; illuminance 99 324; motion_count 118 072; pkt_lost 11 947; pkt_sent 11 947; power_w 198 185; pressure_hpa 11 865; rssi_dbm 11 947; sound_db 59 944; temperature 176 815; vib_rms 117 908*
Призначений стек	*InfluxDB / DuckDB SQL / Matrix Profile / Streamlit*
Точність / повнота (навчальний набір)	*базовий детектор: 0,05 / 0,22; Matrix Profile (після налаштування): 0,36 / 0,44*
Пропускна здатність	*211 207 записів/с (завантаження в InfluxDB 2.7)*

[У таблиці — приклад для навчального набору; впишіть значення свого варіанта.] Значення полів 1–6 скопійовано з виводу `selfcheck.py`, точність і повнота – з підрозділу 3.2, пропускна здатність – з підрозділу 3.1. [Якщо кількість пристроїв у маніфесті і в даних різниться — одним реченням: чому, з посиланням на підрозділ 3.3.]

ЯК ЧИТАТИ ЦЕЙ ЗРАЗОК. Жовтим позначено те, що вписуєте ви: дані свого варіанта, результати своїх вимірювань, знахідки, рисунки. Сірим курсивом — приклади з навчального набору (варіант 0, стек InfluxDB / DuckDB SQL / Matrix Profile / Streamlit): замініть їх своїми значеннями і своїм стеком. Текст без позначок показує обсяг, будову і стиль викладу; пишіть свій, своїми словами, з посиланнями на джерела. Цю примітку видаліть.

ЗМІСТ

ВСТУП....4
РОЗДІЛ 1. Теоретична частина....6
1.1 Архітектура систем інтернету речей....6
1.2 Протоколи передавання телеметрії та їх захищеність....6
1.3 Характеристики великих даних, архітектури обробки....8
1.4 Методи виявлення аномалій у часових рядах....8
1.5 Модель загроз пристроїв інтернету речей....10
РОЗДІЛ 2. Проєктування та реалізація....12
2.1 Обґрунтування архітектури конвеєра....12
2.2 Реалізація вузла та протоколу передавання....13
2.3 Організація зберігання даних....15
2.4 Реалізація детекторів....16
РОЗДІЛ 3. Аналіз результатів....18
3.1 Показники продуктивності конвеєра....18
3.2 Порівняння детекторів за точністю та повнотою....19
3.3 Виявлені інциденти та методика їх встановлення....23
3.4 Недоліки рішення та напрями вдосконалення....25
ВИСНОВКИ....26
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ....28
Додаток 1. Фрагменти програмного коду....31
Додаток 2. Файл findings.json....32
Додаток 3. Знімки екрана дашборда....33

ВСТУП

Кількість пристроїв інтернету речей (IoT) у світі вже обчислюється десятками мільярдів, а розумні будівлі є одним із найпоширеніших сценаріїв їх застосування: давачі температури, вологості, освітленості, руху, енергоспоживання й витоку води безперервно передають телеметрію через шлюзи до сховища, де на її основі ухвалюють рішення про керування інженерними системами [9, 16]. Відмова давача, яку не помітили вчасно, означає хибні керувальні дії, а скомпрометований пристрій – точку входу до внутрішньої мережі: публікація-підписка MQTT без шифрування дозволяє будь-кому прочитати й підробити повідомлення [4, 17]. Тому моніторинг телеметрії має виявляти не лише технічні відмови, а й ознаки несанкціонованої активності.

Специфіка задачі полягає в тому, що навіть невелика будівля за кілька тижнів генерує мільйони записів. Обробка такого масиву «в лоб» засобами електронних таблиць неможлива, а наївне завантаження всього набору в оперативну пам’ять перестає працювати, щойно обсяг даних зростає на порядок. Отже, потрібен конвеєр, що поєднує ефективне зберігання часових рядів, порційну обробку і методи виявлення аномалій, якість яких можна виміряти.

Об’єктом дослідження є телеметрія розподіленої мережі давачів розумної будівлі. Предметом дослідження є методи виявлення відмов обладнання та ознак несанкціонованої активності за даними телеметрії.

Мета роботи – спроєктувати та реалізувати конвеєр збору, зберігання й аналізу телеметрії IoT, здатний у наборі обсягом понад мільйон записів виявляти інциденти дванадцяти класів, і оцінити його продуктивність та якість виявлення.

Для досягнення мети розв’язано такі завдання:

1) реалізувати вузол IoT на мікроконтролері ESP32 у симуляторі Wokwi та дослідити захищеність каналу MQTT без шифрування і з TLS;

2) побудувати конвеєр завантаження набору даних у призначене сховище *InfluxDB* та обробки засобами *DuckDB SQL*, виміряти його продуктивність і межу оперативної пам’яті;

3) реалізувати базовий детектор і призначений детектор на основі *Matrix Profile*, оцінити їхню точність і повноту на навчальному наборі;

4) перевірити детектори на реальних даних Intel Berkeley Research Lab і доопрацювати попередню обробку;

5) виявити інциденти в наборі даних варіанта та обґрунтувати кожну знахідку;

6) побудувати дашборд стану мережі в *Streamlit*.

Новизна роботи полягає в *отриманих власних вимірюваннях для призначеного стеку: порівнянні трьох сховищ на одному наборі даних і одній машині, кількісному зіставленні двох режимів методу Matrix Profile (пошук «незвичних» і «надто схожих» фрагментів) для різних класів інцидентів та в методиці підтвердження знахідок за кількома незалежними ознаками.*

Під час роботи використано систему штучного інтелекту [назва системи і для чого саме] для пояснення повідомлень про помилки в коді конвеєра та перекладу фрагментів англомовної документації. Усі результати вимірювань і знахідки отримано власним конвеєром і перевірено автором.

[Примітка до зразка, у своїй записці видаліть: приклади в таблицях отримано на навчальному наборі (варіант 0) на комп’ютері Apple M1 (8 ядер, 16 ГБ оперативної пам’яті, macOS 26.7).]

РОЗДІЛ 1. Теоретична частина

1.1 Архітектура систем інтернету речей

Еталонна архітектура IoT, визначена стандартом ISO/IEC 30141:2024, розглядає систему як сукупність доменів: фізичних об’єктів, давачів і виконавчих пристроїв; шлюзів і мереж доступу; сервісів зберігання й обробки даних; застосунків і користувачів [1]. Публікація NIST SP 800-183 описує ту саму систему через «примітиви»: давач, агрегатор, канал зв’язку, зовнішня утиліта та рішення, і наголошує, що довіра до результату не може перевищувати довіри до найслабшого примітиву [5]. Для розумної будівлі це означає, що коректність аналітики залежить від стану кожного давача і кожного шлюзу.

У системі, що розглядається в роботі, домен пристроїв утворюють [кількість] давачів [скількох] зон будівлі ([перелік зон з маніфесту]), домен шлюзів – [кількість і перелік шлюзів з маніфесту], які агрегують повідомлення давачів і передають власні службові показники (кількість надісланих і втрачених пакетів, рівень сигналу, вільну пам’ять). Вузол на ESP32, реалізований у підрозділі 2.2, відповідає домену пристроїв, брокер MQTT – домену мережевого обміну, *сховище InfluxDB і обробка DuckDB* – домену сервісів, а детектори й дашборд – домену застосунків.

Особливістю IoT є обмеженість кінцевих пристроїв: малі обсяги пам’яті, живлення від батарей, нестабільний радіоканал. Звідси дві практичні вимоги до архітектури: протокол передавання має бути легким, а аналітичний рівень – стійким до пропусків, дублікатів і неможливих значень, які в реальних мережах є нормою, а не винятком (див. підрозділ 3.2).

1.2 Протоколи передавання телеметрії та їх захищеність

MQTT (Message Queuing Telemetry Transport) – протокол прикладного рівня з моделлю «видавець – підписник», стандартизований OASIS (поточна версія 5.0) [4]. Пристрої не звертаються один до одного напряму: видавець надсилає повідомлення в іменований топік на брокер, а брокер пересилає його всім підписникам цього топіка. Протокол визначає три рівні гарантії доставки (QoS 0 – «не більше одного

разу», QoS 1 – «щонайменше один раз», QoS 2 – «рівно один раз»), прапорець retain, за яким брокер зберігає останнє повідомлення топіка для нових підписників, і механізм Last Will: повідомлення, яке брокер публікує від імені клієнта, якщо той розірвав з'єднання без попередження.

Альтернативами для телеметрії є CoAP – протокол «запит – відповідь» поверх UDP, спроєктований для обмежених пристроїв [6], і HTTP. Порівняння наведено в таблиці 1.

Таблиця 1 – Порівняння протоколів передавання телеметрії

Характеристика	MQTT	CoAP	HTTP/1.1
Модель обміну	видавець – підписник через брокер	запит – відповідь, спостереження (Observe)	запит – відповідь
Транспорт	TCP	UDP	TCP
Мінімальний заголовок	2 Б	4 Б	десятки–сотні байтів
Гарантії доставки	QoS 0, 1, 2	підтверджувані повідомлення (CON)	на рівні TCP
Захист каналу	TLS	DTLS	TLS
Виявлення відключення	Last Will, keep-alive	немає вбудованого	немає вбудованого

Сам протокол MQTT не шифрує даних: ім'я користувача і пароль у пакеті CONNECT та вміст PUBLISH передаються відкритим текстом. Публічні брокери, крім того, не обмежують підписку: клієнт, що підписався на шаблон топіка з символом #, отримує всі повідомлення гілки. Тому захищене розгортання вимагає щонайменше трьох заходів: TLS для конфіденційності й автентифікації брокера, автентифікації клієнтів і списків контролю доступу до топіків [4, 2]. Ці загрози перевірено експериментально в підрозділі 2.2.

Формат корисного навантаження протокол не визначає. Найпоширеніший – JSON: він читається людиною, але надлишковий. CBOR (RFC 8949) – двійкове кодування тієї самої моделі даних, компактніше за рахунок двійкового подання чисел і довжин [3]. Виграш від CBOR виміряно в підрозділі 2.2.

1.3 Характеристики великих даних, архітектури обробки

Великі дані традиційно характеризують «5V»: обсяг (volume), швидкість надходження (velocity), різноманітність (variety), достовірність (veracity) і цінність (value). Телеметрія IoT має всі п’ять ознак: мільйони записів за тижні, безперервний потік, п’ятнадцять різнотипних показників, пропуски й помилкові значення, а цінність – лише за умови своєчасного виявлення відхилень.

Розрізняють пакетну обробку, за якої накопичений масив аналізують цілком, і потокову, за якої кожен запис обробляють у момент надходження. Архітектура Lambda поєднує обидва шляхи, Kappa – обробляє все як потік [8]. У роботі використано спрощений варіант Lambda: пакетний шлях *(набір даних варіанта → InfluxDB → DuckDB → детектори)* і потоковий (вузол → брокер → споживач → файл Parquet), що сходяться на рівні аналізу.

Для аналітичних запитів вирішальним є спосіб зберігання. Рядкові СКБД (SQLite, PostgreSQL) зберігають запис цілком і ефективні для транзакцій, але для агрегування одного стовпця змушені читати всі. Стовпцеві формати (Parquet) і СКБД (DuckDB) зберігають кожен стовпець окремо, що дає змогу читати лише потрібні стовпці і краще стискати однотипні значення. Окремий клас – бази часових рядів (InfluxDB, TimescaleDB): вони індексують записи за часом і тегами, стискають послідовні мітки часу та значення і підтримують політики зберігання. [8]. [Модель зберігання свого сховища — у зразку InfluxDB:] *InfluxDB 2 організує дані як вимірювання (measurement) з тегами (індексованими рядковими атрибутами) і полями (значеннями) та зберігає їх у файлах TSM після накопичення в журналі WAL [22].*

1.4 Методи виявлення аномалій у часових рядах

Аномалією називають спостереження або фрагмент ряду, що суттєво відрізняється від очікуваної поведінки. Розрізняють точкові аномалії (окремий викид), контекстні (значення нормальне саме по собі, але не в цей час) і колективні (незвичною є послідовність значень) [10]. Методи виявлення можна поділити на кілька груп.

Статистичні методи порівнюють значення з оцінкою норми. Класична z-оцінка чутлива до самих викидів, тому для телеметрії застосовують робастний варіант на основі медіани і медіанного абсолютного відхилення (MAD):

$$z = (x - \mathrm{med}) / (1{,}4826 \cdot \mathrm{MAD}), \qquad (1)$$

де med і MAD обчислено в рухомому вікні, а множник 1,4826 узгоджує MAD зі стандартним відхиленням нормального розподілу. До цієї ж групи належать контрольні карти EWMA [18] і узагальнений тест ESD для кількох викидів [19].

Методи декомпозиції (STL [12]) розкладають ряд на тренд, сезонну і залишкову складові й шукають аномалії в залишку, що усуває хибні спрацювання на добовому циклі; поєднання STL з тестом ESD запропоновано для моніторингу хмарних сервісів [13]. Методи на основі щільності (LOF [11]) і ізоляції (Isolation Forest [15]) працюють з векторами ознак і виявляють точки, що лежать в області низької щільності або легко відокремлюються випадковими розбиттями.

[Тут докладно — свій призначений метод: ідея, формула, параметри, переваги й недоліки. У зразку — Matrix Profile:]

Matrix Profile – призначений у варіанті метод [20]. Для ряду T довжини n і довжини фрагмента m матричний профіль P – це вектор, i-й елемент якого дорівнює відстані від фрагмента T[i, m] до найближчого до нього фрагмента деінде в ряді (за винятком тривіальних збігів поруч із i):

$$P[i] = \min_{j:\ |i-j| \geq m/4} d(\hat{T}[i, m],\ \hat{T}[j, m]), \qquad (2)$$

де $\hat{T}$ – z-нормований фрагмент, d – евклідова відстань. Оскільки фрагменти нормуються, метод порівнює форму, а не рівень. Великі значення P (дискорди) означають фрагмент, що не схожий ні на що інше в ряді: сплеск, обрив, небачену форму. Значення, близькі до нуля, означають, що фрагмент майже точно повторює інший: це характерно для «завмерлого» давача і для строго регулярної періодичності. Алгоритм STOMP обчислює профіль за $O(n^2)$ операцій без додаткового параметра порогу [21]; у роботі використано реалізацію бібліотеки stumpy [14]. Перевага методу – єдиний параметр m і здатність знаходити

колективні аномалії; недолік – квадратична складність і нечутливість до повільних змін рівня (дрейфу), бо після нормування фрагмент із дрейфом має ту саму форму.

Якість виявлення оцінюють точністю і повнотою:

$$\text{Precision} = TP / (TP + FP), \quad \text{Recall} = TP / (TP + FN), \tag{3}$$

де TP – кількість вікон, що збіглися з інцидентом, FP – хибні вікна, FN – пропущені інциденти. Вікно вважається збігом, якщо міра перекриття за часом IoU (відношення довжини перетину до довжини об’єднання інтервалів) не менша за 0,25. F1-міра є гармонічним середнім точності й повноти.

1.5 Модель загроз пристроїв інтернету речей

OWASP IoT Top 10 називає серед найпоширеніших вад слабкі або вбудовані паролі, незахищені мережеві сервіси, незахищені інтерфейси екосистеми, відсутність безпечного оновлення та недостатній захист конфіденційності [25]. Стандарт ETSI EN 303 645 формулює базові вимоги до споживчих пристроїв IoT: відсутність універсальних паролів за замовчуванням, захищений зв’язок, мінімізацію відкритих інтерфейсів, стійкість до відмов і моніторинг телеметрії [2].

Для розумної будівлі актуальні загрози трьох груп: відмови обладнання (розряд батарей, «завмирання» давача, обрив зв’язку), помилки конфігурації (переплутані давачі, розсинхронізація годинника) і цілеспрямовані дії порушника. Ознаки компрометації, що розглядаються в роботі, зіставлено з техніками MITRE ATT&CK [24] у таблиці 2.

Таблиця 2 – Класи інцидентів – ознаки компрометації і техніки MITRE ATT&CK

Клас	Що спостерігається в телеметрії	Техніка ATT&CK	Сценарій порушника
rogue_device	пристрій, відсутній у маніфесті, публікує дані	T1200 Hardware Additions	підключення стороннього пристрою до мережі будівлі
beaconing	інтервали між повідомленнями стають строго однаковими	T1071 Application Layer Protocol	заражений пристрій регулярно «відмічається» на сервері керування
exfil_pattern	вихідний трафік шлюзу зростає без зростання телеметрії	T1041 Exfiltration Over C2 Channel	викрадення даних через скомпрометований шлюз
(канал без TLS)	повідомлення і паролі читаються відкритим текстом	T1040 Network Sniffing	перехоплення трафіку MQTT у локальній мережі

Решта класів (stuck_at, outage, spike_storm, drift, clock_skew, energy_anomaly, zone_incoherence, gateway_group_loss, sensor_swap) описують відмови і помилки, але мають значення для безпеки, оскільки можуть маскувати дії порушника або бути їхнім наслідком: наприклад, узгоджена втрата зв’язку всіх пристроїв одного шлюзу може означати як збій живлення, так і атаку на сам шлюз.

РОЗДІЛ 2. Проєктування та реалізація

2.1 Обґрунтування архітектури конвеєра

Склад конвеєра визначено маніфестом варіанта *(сховище InfluxDB, обробка DuckDB SQL, детектор Matrix Profile, дашборд Streamlit)* і вимогами методичних вказівок до двох входів даних – пакетного і потокового. Схему наведено на рис. 1.

[РИСУНОК: схема вашого конвеєра — вузол → брокер → споживач → сховище → обробка → детектори → дашборд, з назвами вашого стеку; намалюйте в draw.io]

Рис. 1. – Схема конвеєра збору та аналітики телеметрії

Пакетний шлях починається з файлу варіанта у форматі Parquet. Сценарій `measure.py` завантажує його в *InfluxDB* і виконує контрольні запити мовою Flux. Для важких агрегацій (добова статистика за кожним пристроєм і показником, кількість повідомлень за годину) використано *DuckDB*: він виконує SQL безпосередньо над файлом Parquet, читає лише потрібні стовпці і не потребує завантаження всього набору в пам’ять. Детектори отримують ряд одного пристрою і одного показника, тому для них важлива швидкість вибірки за пристроєм. Потоковий шлях – вузол ESP32 у Wokwi, брокер Mosquitto і споживач `consumer.py`, що записує прийняті повідомлення у файл того самого формату, що й набір варіанта, тож далі вони обробляються тим самим кодом.

[Чому для вашого обсягу підходять саме ваше сховище й інструмент обробки — у зразку для InfluxDB:] *Вибір InfluxDB для цього завдання виправданий тим, що вся телеметрія є часовими рядами з невеликою кількістю тегів (пристрій, показник), а типові запити – вибірка ряду за інтервалом часу і агрегування у вікнах. Обмеження InfluxDB – повільніше за вбудовані СКБД пакетне завантаження через HTTP API (кількісно – у підрозділі 3.1), тому важкі аналітичні запити передано DuckDB.*

[Один абзац про дашборд (стадія 6): що показує кожна сторінка і чому саме так — наприклад, чому на огляді мережі показано відсоток від звичайного, а не кількість повідомлень.]

2.2 Реалізація вузла та протоколу передавання

Вузол реалізовано на платі ESP32 DevKit у симуляторі Wokwi (рис. 2). До плати підключено цифровий давач температури й вологості DHT22 і фоторезистор у дільнику напруги з резистором 10 кОм; освітленість обчислюється з напруги на вході АЦП. Кожні 10 с вузол публікує три повідомлення у топіки `kursova/<варіант>/<device_id>/<показник>` у форматі JSON (Додаток А методичних вказівок) з полями `device_id`, `metric`, `value`, `ts` і порядковим номером `seq`, за яким споживач виявляє втрачені повідомлення.

[Поясніть формулу перерахунку напруги з АЦП в освітленість: дільник напруги з резистором 10 кОм, опір фоторезистора і степенева залежність опору від освітленості з параметрами GAMMA = 0,7 і RL10 = 50 (TODO 2 посібника).]

[РИСУНОК: знімок Wokwi — схема підключення плати й давачів і монітор порту з повідомленнями, які публікує ваш вузол]

Рис. 2. – Вузол ESP32 у симуляторі Wokwi: схема підключення і монітор послідовного порту

Під час підключення вузол реєструє на брокері Last Will: повідомлення `{"online": false}` з прапорцем retain у топіку `.../status`. Після зупинки симуляції брокер через інтервал keep-alive публікує цей статус, і споживач отримує подію «ОФЛАЙН» (рис. 3) – так конвеєр відрізняє відключений вузол від вузла, якому нічого надсилати.

[РИСУНОК: знімок вебклієнта HiveMQ або виводу consumer.py, де після зупинки симуляції в топіку …/status видно {"online": false}]

Рис. 3. – Повідомлення Last Will у топіку status після зупинки вузла

Для оцінки надлишковості JSON прошивка обчислює розмір того самого повідомлення в CBOR. Результати наведено в таблиці 3.

Таблиця 3 – Розмір повідомлення у форматах JSON і CBOR [приклад]

Показник	JSON, Б	CBOR, Б	Економія, %
temperature	*83*	*64*	*23*
humidity	*80*	*61*	*24*
illuminance	*89*	*64*	*28*

Економія становить [скільки відсотків — з таблиці 3], оскільки більшу частину повідомлення займають назви полів, які CBOR зберігає як рядки. Для масштабу варіанта ([кількість пристроїв], [скільки записів на хвилину]) за 60 діб це близько [ваш розрахунок] ≈ [число] МБ трафіку, які можна заощадити заміною формату, і значно більше – заміною назв полів числовими ключами. Відповідно до вимог вказівок у мережі залишено JSON, а CBOR лише виміряно.

Захищеність каналу досліджено у три кроки. Спершу вузол публікував дані на публічний брокер `broker.hivemq.com` без TLS; сторонній вебклієнт HiveMQ, підписаний на `kursova/#`, без жодних облікових даних отримав усі повідомлення вузла (рис. 4).

[РИСУНОК: знімок вебклієнта HiveMQ, підписаного на kursova/#, з повідомленнями вашого вузла]

Рис. 4. – Повідомлення вузла, прочитані стороннім вебклієнтом на публічному брокері

Далі трафік симуляції записано у форматі PCAP і відкрито у Wireshark: вміст пакетів PUBLISH видно повністю, включно з назвою топіка і значеннями показників (рис. 5).

[РИСУНОК: знімок Wireshark — пакет PUBLISH вашого вузла, у якому видно топік і значення]

Рис. 5. – Пакет MQTT PUBLISH без шифрування у Wireshark

Нарешті, розгорнуто власний брокер Mosquitto з двома слухачами: порт 1883 з автентифікацією за паролем без шифрування і порт 8883 з TLS (сертифікат брокера, підписаний власним центром сертифікації). Вузол на Python (`node.py`) публікував ті самі повідомлення. На порту 1883 Wireshark показав пароль у пакеті CONNECT відкритим текстом; на порту 8883 – лише записи TLS Application Data (рис. 6). Підсумок наведено в таблиці 4.

[РИСУНОК: знімок Wireshark — трафік на порту 8883, де видно лише TLS Application Data]

Рис. 6. – Трафік MQTT через TLS (порт 8883): вміст зашифровано

Таблиця 4 – Результати дослідження захищеності каналу MQTT [приклад]

Спроба порушника	Публічний брокер, без TLS	Власний брокер, порт 1883	Власний брокер, порт 8883 (TLS)
Підписка без пароля	*усі повідомлення*	*відмова: not authorised*	*відмова: not authorised*
Запис трафіку	*топіки і значення відкритим текстом*	*пароль і дані відкритим текстом*	*лише TLS Application Data; видно адреси, час і розміри*

Отже, пароль без шифрування захищає лише від випадкового підписника, але не від перехоплення: порушник у тій самій мережі отримує облікові дані з першого ж пакета CONNECT. TLS із перевіркою сертифіката брокера за власним `ca.crt` забезпечує конфіденційність і захищає від підставного брокера, але не приховує метаданих (адреси, час, розміри повідомлень). Незахищеним у схемі лишається розмежування доступу: будь-який клієнт із чинним паролем читає всі топіки; це усувається списками контролю доступу `acl_file`.

2.3 Організація зберігання даних

Набір даних має чотири стовпці: `ts` (мітка часу UTC, секунди), `device_id`, `metric` і `value`. [Як дані лягають у ваше сховище — у зразку InfluxDB:] *У InfluxDB кожен запис відображено в точку вимірювання `telemetry` бакета `iot` (таблиця 5). Назви пристроїв і показників зроблено тегами, оскільки за ними виконується фільтрація; значення – полем.*

Таблиця 5 – Відображення набору даних у модель InfluxDB [приклад]

Стовпець Parquet	Елемент InfluxDB	Тип	Призначення
ts	*_time*	*мітка часу, с*	*первинний індекс ряду*
device_id	*тег device_id*	*рядок*	*вибірка ряду пристрою*
metric	*тег metric*	*рядок*	*вибірка показника*
value	*поле value*	*float*	*значення показника*

Завантаження виконується порціями по 500 000 рядків через пакетний API запису клієнта `influxdb-client`, тож пам'ять процесу не залежить від розміру файлу. Аналітичні запити DuckDB також читають Parquet порціями (row groups) і лише потрібні стовпці.

Межу оперативної пам’яті оцінено дослідом із масштабуванням: файл, складений з 1, 2 і 4 копій набору, повністю читався в pandas, а пікове використання пам’яті вимірювалося (таблиця 6).

Таблиця 6 – Пам’ять під час читання всього набору в pandas [приклад]

Копій	Записів	Файл Parquet	DataFrame у пам’яті	Пік під час читання	Пік на запис
1×	*1 157 147*	*5,3 МБ*	*16,2 МБ*	*80,1 МБ*	*69,3 Б*
2×	*2 314 294*	*10,8 МБ*	*32,4 МБ*	*145,1 МБ*	*62,7 Б*
4×	*4 628 588*	*21,6 МБ*	*64,8 МБ*	*262,2 МБ*	*56,7 Б*

Пам’ять зростає лінійно, ≈ [число] МБ + [число] Б на кожен запис. За цією залежністю [обсяг RAM вашого комп’ютера] оперативної пам’яті вичерпаються на ≈ [число] млн записів, тобто наборі, приблизно у [число] разів більшому за ваш варіант. Оцінка за одним читанням ([число] Б на запис) дає меншу межу, ≈ [число] млн, бо приписує сталі витрати інтерпретатора кожному запису; для прогнозу використано пряму. Конвеєр цієї межі не досягає: сховище повертає лише ряд одного пристрою, а *DuckDB агрегує* файл без повного завантаження.

2.4 Реалізація детекторів

Базовий детектор (`detect/baseline.py`) обчислює робастну z-оцінку (1) у рухомому вікні попередньої доби і позначає точки з $|z| > 10$. Поріг 10 замість класичних 3 обрано тому, що розподіли показників IoT мають важкі хвости: з порогом 3 детектор спрацьовував щогодини. Позначки, ближчі одна до одної за 1 год, об’єднуються у вікно, а вікна з менш ніж трьох позначок відкидаються як шум.

[Свій призначений детектор: ідея, параметри і чому саме такі — у зразку Matrix Profile:] *Призначений детектор (`detect/methods/matrix_profile.py`) переводить ряд на рівномірну сітку часу (5 хв для показників, 5 хв для інтервалів між повідомленнями), обчислює матричний профіль (2) бібліотекою stumpy і працює у двох режимах. У режимі discords оцінкою є робастна z-оцінка самого профілю, поріг 8. У режимі repeats оцінкою є «регулярність» $1 - P / \sqrt{(2m)}$, де 1 означає точну копію фрагмента, поріг 0,95. Параметри, обрані на навчальному наборі, наведено в таблиці 7.*

Таблиця 7 – Параметри детекторів [приклад]

Параметр	Базовий	*Matrix Profile*	Обґрунтування
Вікно	*24 год*	*2 год; 6 год для power_w*	*інцидент energy_anomaly триває 6–7 год: вікно 2 год ловить лише його краї*
Поріг	*|z| > 10*	*z(P) > 8 / регулярність > 0,95*	*типові значення; зниження додає хибні вікна на добовому циклі*
Режим	–	*discords; repeats для `--signal interval` і для stuck_at*	*регулярність і «завмирання» – це надто схожі фрагменти*
Злиття позначок	*1 год*	*1 год*	*одна подія – одне вікно*
Мінімум позначок	*3*	*3*	*відсікання поодиноких шумових позначок*

Для лічильників шлюзів (`pkt_sent`, `pkt_lost`) обидва детектори аналізують не значення, а швидкість зростання (`--signal rate`), а для виявлення змін ритму передавання – інтервали між повідомленнями (`--signal interval`). Ключ `--evaluate` порівнює вікна з відповідями навчального набору і рахує TP, FP і FN за правилом IoU ≥ 0,25. Фрагмент коду обчислення регулярності наведено в Додатку 1.

РОЗДІЛ 3. Аналіз результатів

3.1 Показники продуктивності конвеєра

Вимірювання виконано на комп’ютері [процесор, кількість ядер, оперативна пам’ять, ОС; версії Python, бібліотек і сховища] на [ваш набір: варіант №, кількість записів]. Час запитів – медіана трьох запусків. Результати для призначеного сховища наведено в таблиці 8.

Таблиця 8 – Показники продуктивності конвеєра *(InfluxDB, навчальний набір)* [приклад]

Показник	Значення
Обсяг у CSV	*44,7 МБ*
Обсяг у Parquet	*5,3 МБ*
Коефіцієнт стиснення CSV / Parquet	*8,5*
Пропускна здатність завантаження	*211 207 записів/с (1 157 147 за 5,48 с)*
Розмір сховища на диску	*10,9 МБ одразу (журнал WAL); 6,3 МБ після ущільнення (TSM)*
Повне сканування (кількість і середнє за кожним показником)	*0,074 с*
Вибірка за одним пристроєм *(co2-01-wor, 9 692 записи)*	*0,038 с*
Пікова пам’ять процесу Python	*292,2 МБ*
Обробка *DuckDB SQL* (добова статистика і погодинні лічильники)	*0,21 с; пікова пам’ять 157 МБ*

Стиснення у [число] раза пояснюється стовпцевою організацією Parquet: однотипні значення стовпця кодуються словником (`device_id`, `metric` мають лише [число] і [число] різних значень), а мітки часу, що монотонно зростають, – дельта-кодуванням. *Розмір InfluxDB одразу після завантаження відображає журнал WAL; через 10 хв без записів InfluxDB ущільнив дані у файли TSM, і розмір зменшився до 6,3 МБ – менше, ніж у DuckDB.*

Для обґрунтування вибору інструментів на тому самому наборі і тій самій машині виміряно також вбудовані сховища DuckDB і SQLite (таблиця 9).

Таблиця 9 – Порівняння сховищ на навчальному наборі [приклад]

Сховище	Завантаження, записів/с	На диску	Повне сканування	Один пристрій
DuckDB	*5 134 092*	*8,4 МБ*	*0,003 с*	*0,003 с*
SQLite	*517 769*	*89,5 МБ*	*0,563 с*	*0,011 с*
InfluxDB 2.7	*211 207*	*6,3 МБ*	*0,074 с*	*0,038 с*

[Що показує порівняння сховищ на вашій машині — у зразку:] *Стовпцевий DuckDB завантажує дані на порядок швидше за рядковий SQLite і займає в 10 разів менше місця, а повне сканування в ньому швидше у ~190 разів, бо читаються лише два стовпці. InfluxDB найповільніший на завантаженні (обмеження HTTP API і побудова індексу серій), але компактний і швидко вибирає ряд за тегами. Для конвеєра варіанта всі показники достатні: завантаження найбільшого варіанта (7,3 млн записів) триватиме менше хвилини, а вибірка ряду – десятки мілісекунд.*

Потоковий шлях перевірено запуском споживача з вузлом у Wokwi: прийнято [число] записів, нерозібраних повідомлень – [число], подій status – [число] ([звідки взялася кожна подія]); пропущених номерів seq – [число], затримка між ts і received_at: медіана [число] с, найбільша [число] с (команда з кроку 2.5 посібника).

3.2 Порівняння детекторів за точністю та повнотою

Детектори оцінено на [число] рядах навчального набору: [число] рядів містять [число] інцидентів, які детектори мали б знаходити за своєю ідеєю, а [ряди] – «здорові» ряди подібних пристроїв. Хибні спрацювання враховано на всіх рядах, зокрема поза інцидентами. Результати базового детектора наведено в таблиці 10.

Таблиця 10 – Базовий детектор на навчальному наборі [приклад]

Ряд (пристрій / показник / сигнал)	Вікон	TP	FP	FN
power-05-mee / power_w	*2*	*1*	*1*	*0*
power-01-mee / power_w	*0*	*0*	*0*	*0*
temp-06-lob / temperature	*0*	*0*	*0*	*2*
light-03-sto / illuminance	*13*	*0*	*13*	*1*
power-04-wor / power_w	*1*	*0*	*1*	*1*
gw-01 / pkt_lost / rate	*3*	*1*	*2*	*0*
gw-02 / pkt_sent / rate	*1*	*0*	*1*	*1*
water-03-mee / flow_lpm / interval	*6*	*0*	*6*	*1*
vibration-06-lob / vib_rms / interval	*12*	*0*	*12*	*1*
Разом	*38*	*2*	*36*	*7*

Точність базового детектора – [TP] / [вікон] = [число], повнота – [TP] / [інцидентів] = [число]. [Що детектор знаходить, а що ні, і чому — у зразку:] *Детектор надійно знаходить серії неможливих значень (`spike_storm`, IoU 0,98) і стрибок втрат на шлюзі. Основне джерело хибних вікон – добовий цикл*

освітленості: ранковий перехід від 0 до тисяч люксів для вікна попередньої доби виглядає як викид, тому на `light-03-sto` *детектор спрацьовував щодня о 03:00–14:00 UTC (13 вікон). Повільний дрейф і заміну давачів на* `temp-06-lob` *він не бачить принципово: рухома медіана «пливе» разом із рядом.*

Matrix Profile спершу запущено з типовими параметрами (режим discords, вікно 2 год), потім – з параметрами таблиці 7 (таблиця 11).

Таблиця 11 – *Matrix Profile*: типові параметри і після налаштування [приклад]

Ряд	Типові: вікон / TP / FP / FN	Налаштування	Після: вікон / TP / FP / FN
power-05-mee / power_w	*2 / 1 / 1 / 0*	–	*2 / 1 / 1 / 0*
power-01-mee / power_w	*0 / 0 / 0 / 0*	–	*0 / 0 / 0 / 0*
temp-06-lob / temperature	*0 / 0 / 0 / 2*	–	*0 / 0 / 0 / 2*
light-03-sto / illuminance	*14 / 0 / 14 / 1*	*repeats*	*6 / 1 / 5 / 0*
power-04-wor / power_w	*3 / 0 / 3 / 1*	*вікно 6 год*	*2 / 1 / 1 / 0*
gw-01 / pkt_lost / rate	*0 / 0 / 0 / 1*	–	*0 / 0 / 0 / 1*
gw-02 / pkt_sent / rate	*0 / 0 / 0 / 1*	–	*0 / 0 / 0 / 1*
water-03-mee / flow_lpm / interval	*0 / 0 / 0 / 1*	–	*0 / 0 / 0 / 1*
vibration-06-lob / vib_rms / interval	*0 / 0 / 0 / 1*	*repeats*	*1 / 1 / 0 / 0*
Разом	*19 / 1 / 18 / 8*		*11 / 4 / 7 / 5*

З типовими параметрами точність [вашого детектора] – [число], повнота – [число]; після налаштування – [число] і [число] відповідно, F1 зросла з [число] до [число]. [Що дало налаштування і чому — у зразку:] *Найбільший внесок дав режим repeats для інтервалів між повідомленнями: на vibration-06-lob метод виявив строго регулярну публікацію (beaconing) з IoU 0,96 без жодного хибного вікна, тоді як базовий детектор на тому самому ряді дав 12 хибних вікон. Збільшення вікна до 6 год для power-04-wor дозволило фрагменту охопити весь інцидент energy_anomaly (рис. 7); з вікном 2 год метод позначав лише його початок і кінець (IoU 0,13 і 0,12).*

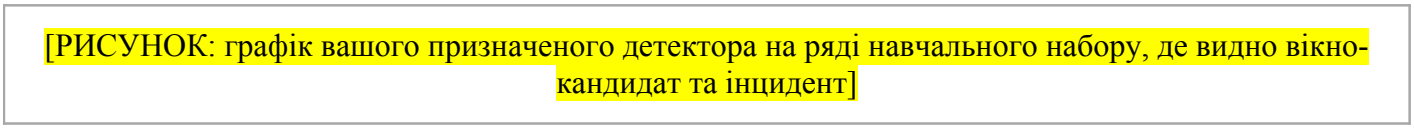


Рис. 7. – *Matrix Profile з вікном 6 год на power-04-wor: вікно-кандидат збігається з інцидентом energy_anomaly*

Залишкові хибні вікна мають зрозумілі причини. На `light-03-sto` *режим repeats позначає нічні години, коли освітленість дорівнює нулю, – для методу*

«завмерлий» давач і темрява однакові. Ці вікна відсікаються перевіркою сусідніх давачів освітленості: вночі нуль показують усі, під час `stuck_at` *– лише один (див. підрозділ 3.3). Пропуски на рядах шлюзів і на* `water-03-mee` *пов'язані з тим, що обрив передавання не утворює точок сітки, а зростання лічильника вдвічі не змінює форми ряду після z-нормування. Для цих класів використано прямі перевірки – інтервали між повідомленнями та відношення лічильників (підрозділ 3.3). Режим repeats для beaconing ілюструє рис. 8.*

[РИСУНОК: ще один графік вашого детектора, наприклад після налаштування параметрів]

Рис. 8. – *Matrix Profile у режимі repeats на інтервалах між повідомленнями vibration-06-lob*

Перевірку на реальних даних виконано на наборі Intel Berkeley Research Lab [23]: [кількість вузлів, тривалість, кількість рядків — зі звіту intel_lab.py]. Виявлені проблеми даних наведено в таблиці 12.

Таблиця 12 – Проблеми реальних даних Intel Lab [приклад]

Проблема	Кількісна ознака	Вплив на детектори
Розряд батарей	*17,5 % показань при напрузі < 2,3 В; 370 196 значень рівно 122,153 °C*	*сотні тисяч неможливих значень на «стелі» давача*
Неможливі значення	*температура до 385 °C, вологість до −8983 %*	*тривоги без реальної події*
Перерви зв'язку	*6 166 перерв понад 10 хв; уся мережа мовчала до 24,5 год*	*вікно детектора порівнює непорівнянні ділянки*
Сторонні вузли	*mote-55…mote-58 і три зіпсовані номери поза mote_locs.txt*	*реальний аналог rogue_device*
Дублікати	*323 збіги «вузол + секунда»; epoch повторюється 113 610 разів*	*подвоєні точки; epoch не може бути ключем*

Попередня обробка складається з семи кроків: видалення дублікатів, фільтрація вузлів за `mote_locs.txt`, відкидання показань при напрузі < 2,3 В, фізичні межі (0–50 °C), заборона вікну детектора перетинати перерви понад 15 хв, 5-хвилинні медіани і видалення тривог, що одночасно виникли на чверті і більше вузлів. Обидва детектори запущено на температурі всіх вузлів після кожного кроку (сценарій `clean.py`); результати наведено в таблиці 13. У клітинках – кількість епізодів, у дужках – частка позначених точок «на смітті», тобто на показаннях, які прибирають кроки 2–4.

Таблиця 13 – Епізоди тривог на даних Intel Lab за кроками обробки [приклад]

Крок обробки	Показань	Базовий	*Matrix Profile, 2 год*	*Matrix Profile, 6 год*
0. Сирі дані	*2 312 781*	*87 (59 %)*	*173 (9 %)*	*164 (18 %)*
1. Без дублікатів	*2 312 463*	*87 (59 %)*	*173 (9 %)*	*164 (18 %)*
2. Лише вузли з mote_locs.txt	*2 302 602*	*84 (59 %)*	*169 (9 %)*	*161 (17 %)*
3. Без показань при напрузі < 2,3 В	*1 899 221*	*60 (14 %)*	*394 (1 %)*	*128 (0 %)*
4. Фізичні межі 0–50 °C	*1 861 732*	*33 (0 %)*	*583 (0 %)*	*125 (0 %)*
5. Вікно не перетинає перерв > 15 хв	*1 861 732*	*31 (0 %)*	*423 (0 %)*	*149 (0 %)*
6. 5-хвилинні медіани	*304 942*	*10 (0 %)*	*360 (0 %)*	*153 (0 %)*
7. Без подій усієї будівлі	*304 942*	*10 (0 %)*	*360 (0 %)*	*153 (0 %)*

Для базового детектора обробка зменшила кількість епізодів з [число] до [число] ([число] %), а частку позначок на «смітті» – з [число] % до [число] %. [Які кроки вплинули найбільше і чому — у зразку:] *Найбільший внесок дали кроки 3–4: до обробки детектор реагував переважно на стрибки температури розряджених вузлів до 122 °C. Крок 6 зменшив епізоди з 31 до 10, бо 5-хвилинні медіани прибирають поодинокі викиди. Крок 7 нічого не змінив: після попередніх кроків жодна тривога не виникала одночасно на чверті вузлів.*

Matrix Profile поводиться інакше. Навіть у сирих даних на «сміття» припадає лише 9 % його позначок: фрагмент «застиглих» 122 °C після z-нормування є сталим, і метод його не позначає (рис. 9, верхня панель). Оскільки метод щоразу позначає найнезвичніші фрагменти кожного ряду, після видалення сміття його увагу перебирають звичайні зміни режиму приміщення, і кількість епізодів зростає з 173 до 360 (+108 %). Отже, для цього методу очищення не зменшує кількість тривог, а змінює те, що позначено; оцінювати його слід за стовпцем «На смітті», а не за кількістю епізодів. З фрагментом 6 год епізодів менше (164 → 153), а частка сміття падає з 18 % до 0 %. Час на рисунку наведено в UTC.

[РИСУНОК: mote-07_temperature.png — вузол mote-07 до і після обробки (python realdata/clean.py --plot mote-07 з ключем --detector вашого детектора)]

Рис. 9. – Вузол mote-07: тривоги *Matrix Profile* до і після обробки, напруга батареї

На рис. 9 видно і межу обробки: 22–24.03 температура `mote-07` *зростає до 50 °C, хоча напруга батареї ще трохи вища за поріг 2,3 В. Це той самий розряд батареї, але Matrix Profile такого плавного зростання не позначає, а поріг напруги його не відсікає. Тому в обробку варіанта перенесено відкидання фізично неможливих значень перед Matrix Profile, а для реальних даних доцільно перевірити суворіший поріг напруги (*`--min-voltage 2.4`*) ціною втрати частини справжніх показань.*

[Аналіз хибних тривог (крок 4.6 посібника): візьміть 3–5 вузлів з найбільшою кількістю тривог після обробки і для кожної тривоги вирішіть, справжня це аномалія чи нормальна поведінка. Аргументуйте часом доби, днем тижня, сусідніми вузлами, напругою батареї, освітленістю. Опишіть, що дала зміна обробки (--min-voltage 2.4, інші показники, інші параметри детектора) і що з цього ви перенесли в обробку свого варіанта (рис. 10).]

[РИСУНОК: вузол з найбільшою кількістю тривог після обробки (python realdata/clean.py --plot mote-NN --detector …); таких рисунків — 3–5]

Рис. 10. – [вузол]: тривоги [вашого детектора] після обробки

3.3 Виявлені інциденти та методика їх встановлення

Пошук інцидентів виконано за класами розділу 7 методичних вказівок: для кожного класу сформульовано ознаку, яку можна перевірити запитом до сховища, і лише потім застосовано детектори для уточнення меж. Для кожної знахідки зафіксовано запит і рисунок, за якими її можна відтворити, що відповідає принципам повторюваності ISO/IEC 27037 [7]. Знахідки, подані у `findings.json` (Додаток 2), наведено в таблиці 14.

Таблиця 14 – Знахідки у варіанті (час UTC) [приклад]

№	Пристрої	Клас	Початок – кінець	Упевн.
1	*door-05-mee*	*rogue_device*	*01.09 21:09 – 03.09 12:07*	*0,95*
2	*power-05-mee*	*spike_storm*	*04.09 00:05 – 04.09 08:30*	*0,95*
3	*light-03-sto*	*stuck_at*	*08.09 02:54 – 08.09 10:14*	*0,90*
4	*water-03-mee*	*outage*	*11.09 09:10 – 12.09 10:35*	*0,90*
5	*vibration-06-lob*	*beaconing*	*10.09 10:50 – 12.09 14:35*	*0,90*
6	*power-04-wor*	*energy_anomaly*	*11.09 19:00 – 12.09 01:40*	*0,86*
7	*gw-01 і 4 давачі за ним*	*gateway_group_loss*	*08.09 17:15 – 09.09 06:10*	*0,80*
8	*gw-02*	*exfil_pattern*	*10.09 05:00 – 11.09 08:15*	*0,60*
9	*light-02-mee*	*clock_skew*	*04.09 14:20 – 08.09 02:20*	*0,55*

[Для КОЖНОЇ знахідки з таблиці 14 — окремий абзац: що саме спостерігається (пристрій, показник, значення, час) і з чим порівняно (звичайна поведінка цього пристрою, сусідні пристрої, маніфест); чому це саме такий клас інциденту; як визначено межі вікна; чому саме така впевненість. До кожної знахідки — рисунок: знімок сторінки «Пристрій» дашборда з вікном знахідки або графік детектора. Нижче — два приклади.]

Знахідка 1. Перелік `device_id` *у даних порівняно з маніфестом: пристрій* `door-05-mee` *відсутній у маніфесті, але з 01.09 21:09 до 03.09 12:07 надіслав 2 334 записи* `door_state`*. Назва імітує схему іменування дверних давачів зони meeting-2, тобто пристрій маскується під легітимний. Межі знахідки – перше й останнє повідомлення.*

Знахідка 2. Протягом 7,3 год давач `light-03-sto` *повертав рівно 45,0 лк (436 показань, стандартне відхилення 0), тоді як інші давачі освітленості в той самий ранок зростали до 2–10 тис. лк (рис. 11). Саме перевірка сусідніх давачів відрізняє цей інтервал від нічних вікон, які позначав Matrix Profile.*

[РИСУНОК до знахідки: знімок сторінки «Пристрій» дашборда з вікном знахідки або графік, на якому видно аномалію]

Рис. 11. – *Освітленість light-03-sto і двох інших давачів: незмінне значення 45 лк уранці 08.09*

[… і так для кожної вашої знахідки.]

3.4 Недоліки рішення та напрями вдосконалення

[Чесно: що не вийшло, які класи ваш пошук не знаходить і чому, що зробили б інакше. У зразку:] *Детектори не виявили повільного дрейфу і заміни давачів (`drift`, `sensor_swap` на `temp-06-lob`) та неузгодженості зони (`zone_incoherence`). Для Matrix Profile дрейф невидимий через z-нормування фрагментів, а базовий детектор «пливе» разом із рядом. Перевірка різниці між давачем і медіаною однотипних давачів зони виявила підозріле відхилення `temp-06-lob`, але без чітких меж, тому знахідку не подано: кожне хибне спрацювання знижує бал на 5 %, і подавати сумнівне вікно недоцільно.*

Точність обох детекторів на навчальному наборі низька (0,05 і 0,36), а повнота не перевищує 0,44. Це означає, що жоден метод не працює як автоматичний «вимикач тривоги»: результати придатні лише як кандидати, які аналітик перевіряє за кількома ознаками. Квадратична складність Matrix Profile (15–20 с на ряд з 4 тис. точок) робить перебір усіх рядів варіанта тривалим; у реальній системі потрібна потокова реалізація або попереднє відсіювання рядів простими правилами.

Напрями вдосконалення: детектор різниці з медіаною зони для дрейфу і заміни давачів; кореляційна перевірка однотипних давачів сусідніх зон для `sensor_swap`; правила узгодженості зони (CO_2, рух і двері) для `zone_incoherence`; списки контролю доступу Mosquitto для розмежування топіків; заміна назв полів числовими ключами для зменшення трафіку.

ВИСНОВКИ

У роботі спроєктовано та реалізовано конвеєр збору, зберігання й аналізу телеметрії розумної будівлі і з його допомогою досліджено набір даних обсягом [число] записів.

1. Реалізовано вузол ESP32 у Wokwi, що публікує три показники через MQTT з механізмом Last Will. Кодування CBOR зменшує повідомлення на [число] %, оскільки більшу частину повідомлення займають назви полів. Експериментально показано, що без TLS повідомлення і пароль перехоплюються з першого пакета, а власний брокер із TLS і автентифікацією залишає порушнику лише метадані з'єднання.

2. Побудовано конвеєр з *InfluxDB і DuckDB SQL*. Parquet зменшує обсяг даних у [число] раза порівняно з CSV; *InfluxDB* приймає [число] тис. записів/с і повертає ряд пристрою за [число] с, *DuckDB* обчислює добову і погодинну статистику всього набору за [число] с. Пам'ять зростає лінійно (≈ [число] Б на запис), і [RAM] ГБ вистачить для ≈ [число] млн записів у разі повного читання; конвеєр цієї межі не досягає завдяки вибірці на рівні сховища.

3. Базовий детектор на навчальному наборі досяг точності [число] і повноти [число]; *Matrix Profile* після налаштування – [число] і [число] (F1 зросла з [число] до [число]). *Режим пошуку надто схожих фрагментів виявив beaconing з IoU 0,96 без хибних вікон.*

4. На реальних даних Intel Lab виявлено [число] % показань від розряджених батарей, [число] перерв зв'язку і сторонні вузли; семикрокова обробка зменшила кількість епізодів тривог базового детектора з [число] до [число] ([число] %), а для *Matrix Profile* звела частку позначок на некоректних показаннях з [число] % до [число] %.

5. У варіанті подано [число] знахідок [скількох] класів, з них [скільки] – ознаки компрометації ([які]); кожну підтверджено щонайменше двома незалежними ознаками. Не виявлено інцидентів класів [яких класів].

6. Побудовано дашборд *Streamlit* з оглядом мережі, деталізацією за пристроєм і часовою шкалою аномалій (Додаток 3).

Практичне значення роботи полягає в тому, що для моніторингу IoT недостатньо одного детектора: метод має відповідати класу інциденту, а знахідка – підтверджуватися перевіркою, незалежною від детектора.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1 ISO/IEC 30141:2024. Internet of Things (IoT) – Reference architecture. Geneva : ISO, 2024. 64 p.

2 ETSI EN 303 645 V3.1.3 (2024-09). Cyber Security for Consumer Internet of Things: Baseline Requirements. Sophia Antipolis : ETSI, 2024. 40 p.

3 Bormann C., Hoffman P. Concise Binary Object Representation (CBOR) : RFC 8949. IETF, 2020. 66 p.

4 MQTT Version 5.0 : OASIS Standard. 07 March 2019. URL: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html (дата звернення: 01.11.2026).

5 Voas J. Networks of ‘Things’ : NIST Special Publication 800-183. Gaithersburg : NIST, 2016. 30 p.

6 Shelby Z., Hartke K., Bormann C. The Constrained Application Protocol (CoAP) : RFC 7252. IETF, 2014. 112 p.

7 ISO/IEC 27037:2012. Information technology – Security techniques – Guidelines for identification, collection, acquisition and preservation of digital evidence. Geneva : ISO, 2012. 38 p.

8 Kleppmann M. Designing Data-Intensive Applications. Sebastopol : O’Reilly Media, 2017. 616 p.

9 Al-Fuqaha A., Guizani M., Mohammadi M., Aledhari M., Ayyash M. Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. IEEE Communications Surveys & Tutorials. 2015. Vol. 17, no. 4. P. 2347–2376.

10 Blázquez-García A., Conde A., Mori U., Lozano J. A. A Review on Outlier/Anomaly Detection in Time Series Data. ACM Computing Surveys. 2021. Vol. 54, no. 3. Article 56.

11 Breunig M. M., Kriegel H.-P., Ng R. T., Sander J. LOF: Identifying Density-Based Local Outliers. Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. Dallas, 2000. P. 93–104.

12 Cleveland R. B., Cleveland W. S., McRae J. E., Terpenning I. STL: A Seasonal-Trend Decomposition Procedure Based on Loess. Journal of Official Statistics. 1990. Vol. 6, no. 1. P. 3–73.

13 Hochenbaum J., Vallis O. S., Kejariwal A. Automatic Anomaly Detection in the Cloud Via Statistical Learning. arXiv:1704.07706. 2017. 13 p.

14 Law S. M. STUMPY: A Powerful and Scalable Python Library for Time Series Data Mining. Journal of Open Source Software. 2019. Vol. 4, no. 39. Article 1504.

15 Liu F. T., Ting K. M., Zhou Z.-H. Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining. Pisa, 2008. P. 413–422.

16 Minoli D., Sohraby K., Occhiogrosso B. IoT Considerations, Requirements, and Architectures for Smart Buildings – Energy Optimization and Next-Generation Building Management Systems. IEEE Internet of Things Journal. 2017. Vol. 4, no. 1. P. 269–283.

17 Neshenko N., Bou-Harb E., Crichigno J., Kaddoum G., Ghani N. Demystifying IoT Security: An Exhaustive Survey on IoT Vulnerabilities and a First Empirical Look on Internet-Scale IoT Exploitations. IEEE Communications Surveys & Tutorials. 2019. Vol. 21, no. 3. P. 2702–2733.

18 Roberts S. W. Control Chart Tests Based on Geometric Moving Averages. Technometrics. 1959. Vol. 1, no. 3. P. 239–250.

19 Rosner B. Percentage Points for a Generalized ESD Many-Outlier Procedure. Technometrics. 1983. Vol. 25, no. 2. P. 165–172.

20 Yeh C.-C. M., Zhu Y., Ulanova L. et al. Matrix Profile I: All Pairs Similarity Joins for Time Series. 2016 IEEE 16th International Conference on Data Mining. Barcelona, 2016. P. 1317–1322.

21 Zhu Y., Zimmerman Z., Senobari N. S. et al. Matrix Profile II: Exploiting a Novel Algorithm and GPUs to Break the One Hundred Million Barrier for Time Series Motifs and Joins. 2016 IEEE 16th International Conference on Data Mining. Barcelona, 2016. P. 739–748.

22 InfluxDB OSS v2 Documentation : InfluxDB storage engine. URL: https://docs.influxdata.com/influxdb/v2/ (дата звернення: 01.11.2026).

23 Intel Berkeley Research Lab sensor data. URL: https://db.csail.mit.edu/labdata/labdata.html (дата звернення: 01.11.2026).

24 MITRE ATT&CK : Enterprise Matrix. Techniques T1040, T1041, T1071, T1200. URL: https://attack.mitre.org/ (дата звернення: 01.11.2026).

25 OWASP Internet of Things Top 10 2018. URL: https://owasp.org/www-project-internet-of-things/ (дата звернення: 01.11.2026).

[Додайте свої джерела; разом — не менше 20, серед них набір Intel Lab, чинні стандарти і статті IEEE. Оформлення — за ДСТУ 8302:2015.]

Додаток 1

Фрагменти програмного коду

[Фрагменти вашого коду, кожен до 10–15 рядків. У зразку:] Обчислення полів картки результатів запитами DuckDB до файлу варіанта:

```
import duckdb, hashlib, json
con = duckdb.connect()
src = "read_parquet('tuning.parquet')"
rows = con.sql(f"SELECT count(*) FROM {src}").fetchone()[0]
observed = con.sql(f"SELECT count(DISTINCT device_id) FROM {src}").fetchone()[0]
per_metric = con.sql(f"SELECT metric, count(*) FROM {src} "
                     "GROUP BY metric ORDER BY metric").fetchall()
manifest = json.load(open("tuning.json"))
ids = sorted(d["device_id"] for d in manifest["devices"])
sha = hashlib.sha256("\n".join(ids).encode("utf-8")).hexdigest()
```

Перевірка регулярності публікації (клас beaconing): інтервали між повідомленнями пристрою за годину та їх розкид:

```
SELECT device_id,
       date_trunc('hour', to_timestamp(ts))       AS hour,
       count(*)                                    AS n,
       stddev_samp(dt)                             AS dt_std
FROM (SELECT device_id, ts,
             ts - lag(ts) OVER (PARTITION BY device_id, metric ORDER BY ts) AS dt
      FROM read_parquet('tuning.parquet'))
WHERE dt IS NOT NULL
GROUP BY ALL
HAVING n > 20 AND dt_std < 0.5
ORDER BY device_id, hour;
```

Повний код конвеєра, детекторів і дашборда – в архіві [Прізвище_NNN_код.zip], зданому разом із пояснювальною запискою.

Додаток 2

Файл findings.json

[Ваш findings.json повністю, як його перевірив selfcheck.py. У зразку — початок файла навчального набору:]


```
{
  "variant": 0,
  "fingerprint": {
    "rows": 1157147,
    "devices_observed": 52,
    "device_list_sha256":
"1a33058d845b45c2ce0ddff78318cad7cab1f1b8c9a25aed2e3cdbafc84b7098"
  },
  "findings": [
    {
      "devices": [
        "door-05-mee"
      ],
      "class": "rogue_device",
      "start": "2025-09-01T21:09:00Z",
      "end": "2025-09-03T12:07:00Z",
      "confidence": 0.95,
      "evidence": "Пристрою немає в маніфесті; 2 334 записи door_state"
    },
    {
      "devices": [
        "power-05-mee"
      ],
      "class": "spike_storm",
      "start": "2025-09-04T00:05:00Z",
      "end": "2025-09-04T08:30:00Z",
      "confidence": 0.95,
      "evidence": "73 показання потужності поза діапазоном 0–12 кВт (до 132 кВт)"
    },
…
  ]
}
```

Додаток 3

Знімки екрана дашборда

[РИСУНОК: знімок сторінки «Огляд мережі» вашого дашборда]

Рис. 12. – Дашборд *Streamlit*: огляд стану мережі

[РИСУНОК: знімок сторінки «Пристрій» вашого дашборда]

Рис. 13. – Дашборд *Streamlit*: деталізація за пристроєм

[РИСУНОК: знімок сторінки «Аномалії» вашого дашборда]

Рис. 14. – Дашборд *Streamlit*: часова шкала аномалій